Kuis Formatif

Nama : Hikmah Azizah

NIM : 2225250086

Kelas : 3B

1. Apa perbedaan syntax error, runtime error, dan logical error?

Jawab: *Syntax Error* (Kesalahan Sintaks): Kesalahan penulisan aturan bahasa pemrograman (misalnya kurang titik koma ;, salah eja perintah, atau kurung kurawal yang tidak seimbang). Kesalahan ini langsung terdeteksi oleh *compiler* atau *interpreter* sebelum program dijalankan. *Runtime Error* (Kesalahan Saat Eksekusi): Kesalahan yang terjadi ketika program sedang berjalan. Program biasanya langsung terhenti (*crash*). Contohnya adalah pembagian dengan nol atau mengakses indeks array yang tidak ada. *Logical Error* (Kesalahan Logika): Program berjalan lancar tanpa *crash*, tetapi menghasilkan output yang salah karena alur logika atau rumus yang ditulis oleh programmer keliru.

2. Apa fungsi traceback?

Jawab: *Traceback* berfungsi untuk melacak dan menampilkan jalur pemanggilan fungsi (*call stack*) saat terjadi *runtime error*. Informasi ini membantu developer mengetahui baris kode, file, serta urutan fungsi spesifik yang menyebabkan program mengalami kegagalan.

3. Kapan Step Into digunakan?

Jawab: *Step Into* digunakan ketika kita sedang melakukan *debugging* dan ingin masuk ke dalam sebuah fungsi atau *method* yang dipanggil pada baris kode saat itu untuk memeriksa setiap baris instruksi di dalamnya secara mendetail.

4. Apa beda Step Over dan Step Into?

Jawab: *Step Over*: Mengeksekusi baris kode saat ini sebagai satu kesatuan utuh. Jika ada pemanggilan fungsi di baris tersebut, fungsi akan dijalankan sampai selesai tanpa masuk ke dalam kodenya, lalu debugger langsung pindah ke baris berikutnya. *Step Into*: Masuk ke dalam detail baris kode dengan cara menyelami isi fungsi atau *method* yang sedang dipanggil untuk memeriksa proses di dalamnya baris demi baris.

5. Mengapa nilai batas perlu dimasukkan dalam test case?

Jawab: Nilai batas (*boundary values*) penting karena sebagian besar *bug* sering kali terjadi di titik transisi (misalnya pas di angka minimum, maksimum, atau tepat di luar rentang yang diizinkan). Mengujinya memastikan sistem menangani kondisi tepi (*edge cases*) dengan benar.

6. Apa beda expected result dan actual result?

Jawab: *Expected Result* (Hasil yang Diharapkan): Output atau perilaku sistem yang seharusnya terjadi berdasarkan dokumen spesifikasi atau kebutuhan fungsional. *Actual Result* (Hasil Aktual): Output atau perilaku nyata yang benar-benar terjadi saat program dijalankan atau diuji. Jika keduanya tidak cocok, berarti ditemukan *bug*.

7. Apa yang dimaksud regresi setelah perbaikan?

Jawab: Regresi adalah kondisi di mana suatu fitur atau bagian sistem yang tadinya berfungsi normal mendadak rusak atau mengalami penurunan performa akibat adanya perubahan kode atau perbaikan *bug* baru yang tidak sengaja memengaruhi bagian lain.

8. Mengapa debugging perlu diawali dengan reproduksi masalah?

Jawab: Reproduksi masalah (*reproduction*) memastikan bahwa developer dapat memicu kemunculan *bug* secara konsisten. Tanpa bisa mereproduksinya, sulit untuk memverifikasi apakah perbaikan yang dilakukan benar-benar menyelesaikan akar masalah atau hanya kebetulan.

9. Mengapa perubahan kode sebaiknya minimal saat memperbaiki bug?

Jawab: Prinsip ini bertujuan untuk mengurangi risiko menimbulkan *bug* baru (regresi), menjaga kompleksitas kode tetap rendah, serta memudahkan proses *code review* agar rekan tim bisa fokus sepenuhnya pada akar masalah tanpa gangguan perubahan yang tidak relevan.

10. Apa manfaat commit khusus bertipe fix pada GitHub?

Jawab: *Commit* bertipe fix (mengikuti konvensi seperti *Conventional Commits*) memberikan manfaat:

- Mempermudah pelacakan riwayat perubahan (*changelog generation* otomatis).
- Memperjelas maksud *commit* bagi anggota tim saat melakukan *code review*.
- Membantu alat otomatisasi CI/CD dalam menentukan versi rilis perangkat lunak (misalnya menaikkan nomor versi *patch*).